Predictive Storage Caching and Tiered Data Migration

Nilay Borkar: 2023A7PS0401G
Ishan Kaustubh Mehta: 2023A7PS0379G

The problem

With advancement in technology, the compute time for AI/ML training and tasks have greatly reduced.

The 4 major steps in ML training are:

- Loading the data from the storage
- Forward Computing
- Backpropagation
- Sharing the updated weights with other nodes.

During training the Model needs a lot of data, but this data is in storage and the time it takes to fetch this data causing I/O to act as a very big bottleneck.

The Problem

There is an upper bound on the amount of the Memory available to us, and this memory is not enough to hold all the training data which can be anywhere from terabytes to petabytes.

Thus, we need I/O operations. If we don't have the data for the compute it sits ideal, which is not good.

There is some storage available which is very fast however this is very expensive and can only hold a very small percentage of the training data.

Since, the training data has to be random as we don't want the Model to overfit by recognising a pattern. Thus, data fetching operations are reactive, meaning the data is fetched from storage after the Model requests it.

The data also doesn't necessarily follow the principles of temporal or spatial locality since the data fetched is random.

Some Important Terms

Nodes: The “blocks” which make up the supercomputer, they are independent and connected with the others

PFS(Parallel File system): A storage system that spreads data across multiple storage servers, allowing multiple clients (GPUs/Nodes) to access the *same* file simultaneously.

RDMA: It allows one computer to access the memory of another computer directly, without involving the operating system or CPU of either machine.

GPUDirect Storage: A technology that creates a direct data path between storage (like NVMe SSDs) and GPU memory.

SmartNIC: A network card equipped with its own programmable processor. It can perform tasks like packet filtering or encryption directly on the card.

FedCaSe

FedCaSe does not utilize a monetary cost model but instead focuses on minimizing system resource usage and training time while maximizing model accuracy.

The framework treats training time and model error as the primary costs to reduce, specifically targeting the I/O bottleneck to minimize time of training rounds by prioritizing clients and samples in fast memory.

Simultaneously, it aims to maximize training quality by selecting ”experienced” samples that contribute most to loss reduction.

A Reverse Optimization (RO) model balances these costs by taking target reductions in round duration and increases in accuracy to predict the optimal ratio of experienced to novice clients.

The scheduler uses a weighted formula for I/O utility and Model utility to calculate a client’s experience, allowing the system to trade off between speed and accuracy

HVAC

Most of the data accesses which happen during training are Reads, so HVAC proposes a POSIX compliant client-server model with caching for reads, which doesn't need any changes to the existing deep learning pipeline

It has a client which hijacks oper, close or read requests and then sends it to the server, now this server uses a hash to decide which node will have that data, and send a request to that server through the mercury RPC library.

Now that server checks whether, this data is available in the cache at that node, if it is it sends it back, if not then it loads it from the PFS, sends then data and caches it.

Since there is an aggregate cache in HVAC, there is a lot of cache storage available to us. Currently HVAC uses a random eviction policy, this can be changed.

SHADE

The Compute-Storage Discrepancy: While the GPU throughput has increased exponentially, the remote storage bandwidth remain with significant constraints. This leads to **Data Stalls** where I/O can't feed model fast enough.
Failure of Traditional Caching: Most DL Training rely on **Uniform Random Sampling**. Caching policies like LRU or LFU rely on temporal locality. But the data is random & these policies provide a near to zero cache hit ratio.
The Gap SHADE identifies: Caching research has treated all data equally. It ignored the **semantic value** of the data, treating hard-to-learn data same as redundant or already-learned ones.

Innovation: SHADE introduces Priority-based Adaptive Sampling (**PADS**). It gives more importance to the samples with higher losses or uncertainty. By revisiting these "hard" samples more, SHADE creates a temporal locality which the cache exploits.

- **Ranked-based Importance:** Instead of raw losses, SHADE uses relative ranking system to ensure the cache replacement policy remains stable even when the model's overall loss decreases.
- **Ghost Cache:** A metadata cache is maintained to store the importance scores of all the data ever sampled. This prevents "hard-to-learn" data to be forgotten when it is evicted from the memory cache.

SHADE increases the cache hit ratio by upto **4.5x** than LRU. It achieves a throughput of **3.3x in convergence**.

Neuro-Tiering: Predictive Data Migration for Heterogeneous Storage Tiers in DL Training

1. The Core Problem & Options

The Baseline Problem: Deep Learning (DL) training is “starving.” GPUs wait for data from slow remote storage. Standard caching (LRU) is useless because DL data access is randomly shuffled every epoch.

Current Solution (SHADE): It uses “Importance Sampling” to cache high-loss data in RAM. The Gap: It is binary (Cache vs. Disk). It doesn’t use middle tiers like NVMe.

Our Option: We move from Reactive Caching (deciding after a miss) to Predictive Migration (moving data before the GPU asks for it) across multiple tiers (HBM ➔ NVMe ➔ HDD).

Tech

Component	Technology	Why?
Language	Python 3.x	Best support for PyTorch and ML simulation.
Framework	PyTorch 2.x	Standard for DL training research.
ML Engine	PyTorch / Scikit-learn	To build and train the LSTM.
Simulator	SimPy, LibCache Sim	Mimics HBM, NVMe, and HDD latencies.

We will be doing a Trace-Driven Simulation, which is easier than simulating it in real time.

Milestone 2: Trace Generation
Goal: Capture how "Importance" changes over time.
Steps: 1. Instrument a PyTorch loop. 2. As the model trains on a dataset (e.g., CIFAR-100 or Tiny-ImageNet), log the SampleID, LossScore, and Timestamp for every image.
We use ResNet-18 or ResNet-50 because their I/O behavior is well-documented, giving us a strong baseline.

Milestone 3: The Predictive Engine
Goal: Build a model that predicts future importance based on past epochs.
The Tech: LSTM (Long Short-Term Memory).
Importance follows a sequence (High ➔ Medium ➔ Low). LSTMs are good at finding patterns in time series.
Transformers are too heavy for a storage controller.

Output: A model that takes [Loss in Epoch 1, 2, 3] and outputs Predicted Loss in Epoch 4.

Milestone 3 & 4: Tiered Migration & Evaluation
Goal: Demonstrate simulated speedups.
Plug the LSTM predictions into our Multi-Tier Simulator.
Simulation logic: The samples are put into tiers of cache based on their predicted loss.
Final Test: Calculate "Total Training Time" for our policy vs. standard SHADE and LRU.

Milestone 4: Comparison & Evaluation

LRU (Least Recently Used): The standard "dumb" caching used in most systems.
SHADE (Binary Importance): The paper we just read. It only has "In RAM" or "On Disk".
Oracle (Belady's MIN): An "impossible" policy that knows the future perfectly. This shows how close your LSTM gets to perfection.

Key Metrics to Present:
Cache Hit Ratio: Percentage of data found in the "Fast" (HBM) tier.
Mean Load Latency: Average time a GPU waits for a sample.
Epoch Speedup: How much faster one pass of the dataset becomes.